

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) ¹
Student Name	Sadhana J
Register Number	3122247001052
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	01.09.2026
Github Repository	Click Here

1 Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of ten machine learning classifiers – SVM, Naive Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XG-Boost, and Stacking. Each model is trained and validated both without PCA (original feature space) and with PCA (reduced feature space), using hyperparameter tuning and 5-fold cross-validation in both cases.

2 Problem Statement

Given the Wisconsin Diagnostic Breast Cancer dataset with 30 correlated numerical features, evaluate whether reducing dimensionality via PCA improves, worsens, or has negligible effect on classification accuracy, F1-score, and cross-validation stability across a diverse set of linear, instance-based, tree-based, and ensemble classifiers.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer Dataset (wdbc.data)
Dataset Source	UCI ML Repository, https://archive.ics.uci.edu
Number of Samples	569
Number of Features	30 numerical attributes (mean, standard-error, and worst values of 10 nuclear characteristics); ID column dropped as identifier
Number of Classes	2 (Diagnosis: M = Malignant, B = Benign)
Class Distribution	Benign (B) majority, Malignant (M) minority – moderate imbalance
Missing Values	None
Preprocessing	Label encoding of target; StandardScaler applied to all numeric features
Train-Test Split	80% train / 20% test, stratified, random_state=42

4 Exploratory Data Analysis

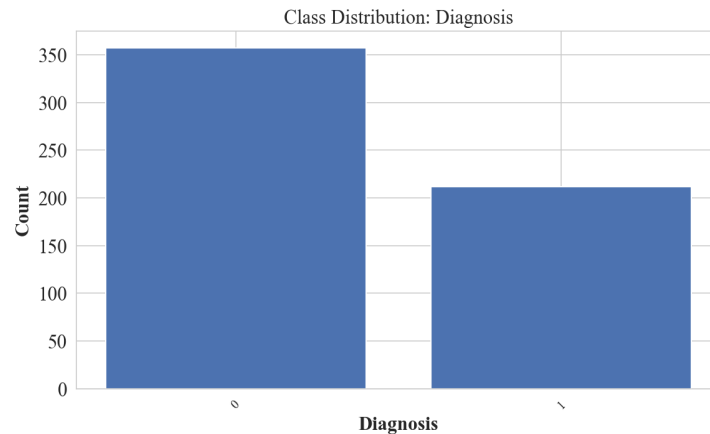


Figure 1: Class Distribution: Diagnosis

Inference: Benign cases outnumber malignant cases, giving a moderate class imbalance, which is why weighted precision/recall/F1 and stratified splitting are used throughout this experiment.

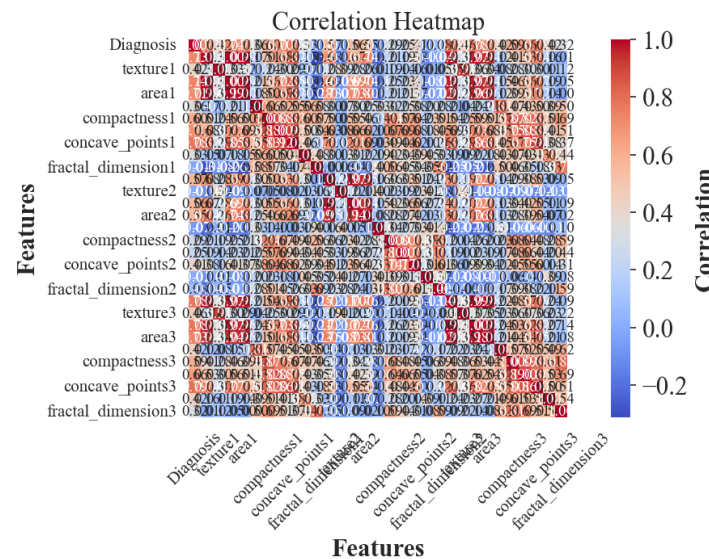


Figure 2: Correlation Heatmap

Inference: The mean, standard-error, and "worst" versions of radius, perimeter, and area are extremely highly correlated with each other, indicating strong feature redundancy – exactly the kind of structure PCA is designed to compress into fewer, uncorrelated components.

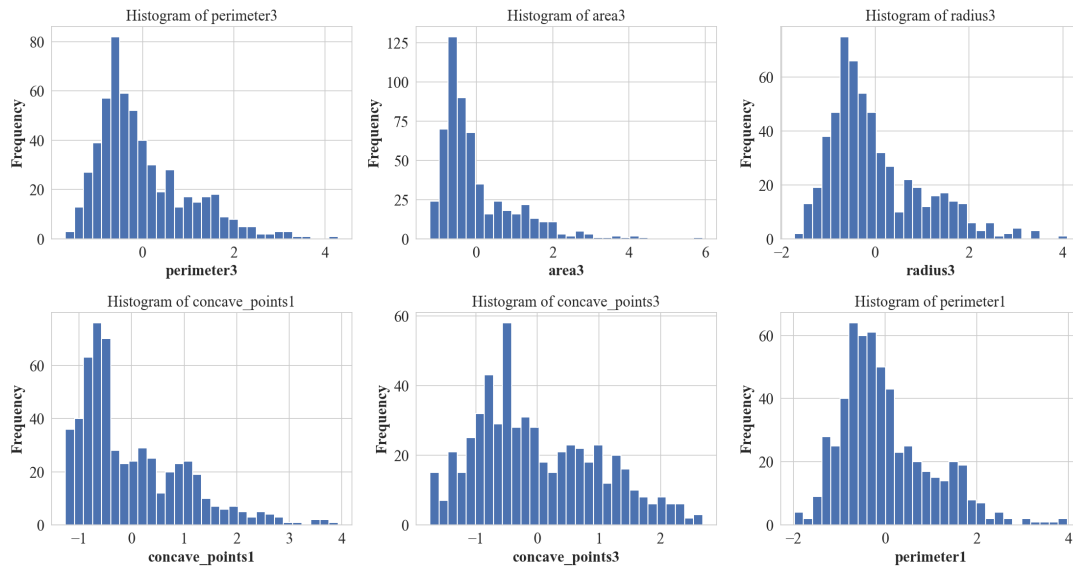


Figure 3: Histogram grid of the top selected features

Inference: Features such as perimeter1 and area1 show a right-skewed distribution with malignant cases trending toward larger values, reinforcing the need for standardization before PCA (which is variance-sensitive).

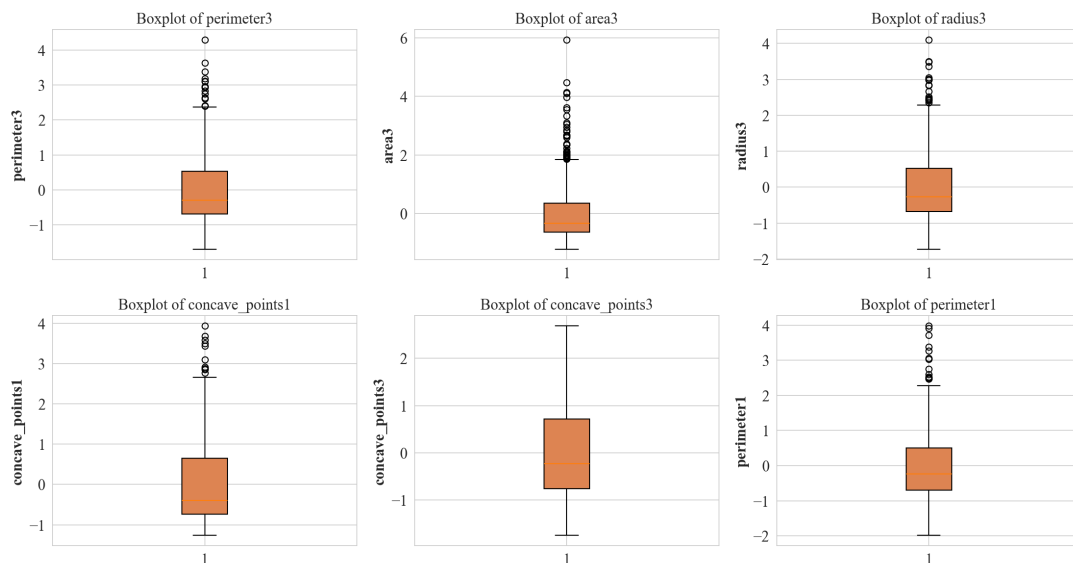


Figure 4: Boxplot grid of the top selected features

Inference: Several features show a noticeable number of high-value outliers from malignant tumors, which could disproportionately influence the direction of the first few principal components if left unscaled.

5 PCA Design and Summary

PCA was applied with `n_components=0.95` (a 95% cumulative explained-variance target) rather than a fixed component count, letting the data itself determine how many components are needed to retain most of the information.

Setting	Components / Variance Target	Explained Variance (%)
With-PCA	95% variance target $\rightarrow$ 10 components retained (from 30 original features)	95.23%

Justification: A 95% variance target was chosen as a standard, widely-used threshold that retains almost all the information in the original 30 features while achieving a $3\times$ reduction in dimensionality ($30 \rightarrow 10$), making it a reasonable trade-off between compression and information loss.

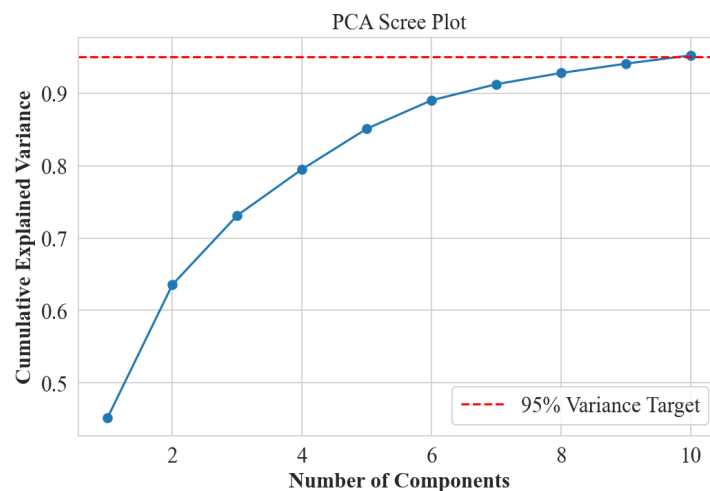


Figure 5: PCA Scree Plot

Inference: The cumulative explained-variance curve rises steeply for the first few components and flattens out after around component 7-8, crossing the 95% threshold at 10 components – confirming that most of the dataset’s variance is concentrated in a small number of directions, consistent with the strong inter-feature correlation observed in the heatmap.

6 Machine Learning Algorithms and Hyperparameter Grids

Model	Hyperparameter Grid
SVM	kernel=rbf, $C \in \{1, 10, 100\}$, $\gamma \in \{\text{scale}, \text{auto}\}$
Naive Bayes	var_smoothing $\in \{10^{-9}, 10^{-8}, 10^{-7}\}$
KNN	n_neighbors $\in \{3, 5, 7, 9\}$, weights $\in \{\text{uniform}, \text{distance}\}$
Logistic Regression	$C \in \{0.1, 1, 10\}$
Decision Tree	max_depth $\in \{3, 5, 7, \text{None}\}$
Random Forest	n_estimators $\in \{50, 100, 150\}$
AdaBoost	n_estimators $\in \{50, 100\}$, learning_rate $\in \{0.1, 1.0\}$
Gradient Boosting	n_estimators $\in \{50, 100\}$, learning_rate $\in \{0.05, 0.1\}$

XGBoost	$n_estimators \in \{50, 100\}$, $max_depth \in \{3, 5\}$
Stacking	Base models: SVM, Naive Bayes, Decision Tree; Meta-learner: Logistic Regression; tuned $final_estimator_C \in \{0.1, 1, 10\}$

All models were tuned using `GridSearchCV` ($cv=5$, $scoring='accuracy'$), separately on the No-PCA and With-PCA feature sets, so each setting gets its own independently optimal hyperparameters.

7 Experimental Setup

Python Version	
Scikit-Learn Version	
XGBoost Version	
IDE	Jupyter Notebook
Operating System	
Hardware	

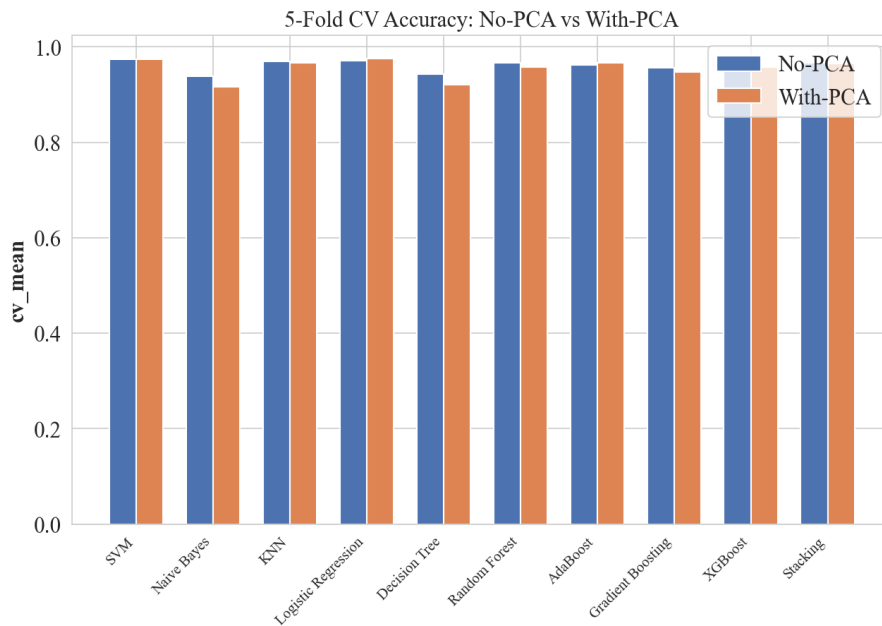
8 Results

Table: Best Hyperparameters and Test Performance

Model	Best Params (No-PCA)	Best Params (With-PCA)	Test Acc (No-PCA)	Test Acc (PCA)	Test F1 (No-PCA)	Test F1 (PCA)
SVM	$C=1$, $\gamma=scale$, $kernel=rbf$	$C=1$, $\gamma=scale$, $kernel=rbf$	0.9737	0.9649	0.9630	0.9500
Naive Bayes	$var_smoothing=1e-9$	$var_smoothing=1e-9$	0.9211	0.8947	0.8889	0.8571
KNN	$n_neighbors=3$, $weights=uniform$	$n_neighbors=3$, $weights=uniform$	0.9386	0.9474	0.9114	0.9250
Logistic Regression	$C=1$	$C=1$	0.9649	0.9825	0.9512	0.9756
Decision Tree	$max_depth=7$	$max_depth=3$	0.9386	0.9123	0.9136	0.8810
Random Forest	$n_estimators=50$	$n_estimators=100$	0.9737	0.9386	0.9630	0.9157
AdaBoost	$lr=1.0$, $n_estimators=50$	$lr=1.0$, $n_estimators=50$	0.9825	0.9474	0.9756	0.9250
Gradient Boosting	$lr=0.1$, $n_estimators=50$	$lr=0.05$, $n_estimators=100$	0.9561	0.9474	0.9383	0.9250
XGBoost	$max_depth=3$, $n_estimators=50$	$max_depth=3$, $n_estimators=50$	0.9649	0.9561	0.9500	0.9383
Stacking	$final_estimator_C=10$	$final_estimator_C=10$	0.9707	0.9649	0.9639	0.9500

Table 5: 5-Fold Cross-Validation Results (No-PCA vs With-PCA)

Model	F1	F2	F3	F4	F5	Avg (NoPCA)	Avg (PCA)
SVM	0.9670	1.0000	0.9670	0.9560	0.9780	0.9736	0.9736
Naive Bayes	0.9231	0.9890	0.9231	0.9231	0.9341	0.9385	0.9165
KNN	0.9780	1.0000	0.9451	0.9780	0.9451	0.9692	0.9670
Logistic Regression	0.9670	0.9890	0.9560	0.9780	0.9670	0.9714	0.9758
Decision Tree	0.9780	0.9670	0.9121	0.9670	0.8901	0.9429	0.9209
Random Forest	1.0000	0.9890	0.9341	0.9890	0.9231	0.9670	0.9582
AdaBoost	0.9780	0.9890	0.9231	0.9890	0.9341	0.9626	0.9670
Gradient Boosting	1.0000	1.0000	0.9231	0.9451	0.9121	0.9560	0.9473
XGBoost	0.9890	0.9890	0.9341	0.9780	0.9560	0.9692	0.9582
Stacking	0.9670	1.0000	0.9560	0.9670	0.9451	0.9670	0.9648

**Figure 6: 5-Fold CV Accuracy: No-PCA vs With-PCA**

Inference: Most models show only a small change in cross-validated accuracy after PCA – SVM is virtually unchanged, Logistic Regression and AdaBoost actually improve slightly with PCA, while Naive Bayes, Decision Tree, Random Forest, Gradient Boosting and XGBoost all drop modestly, indicating tree-based/ensemble models lose some of the fine-grained feature information PCA compresses away.

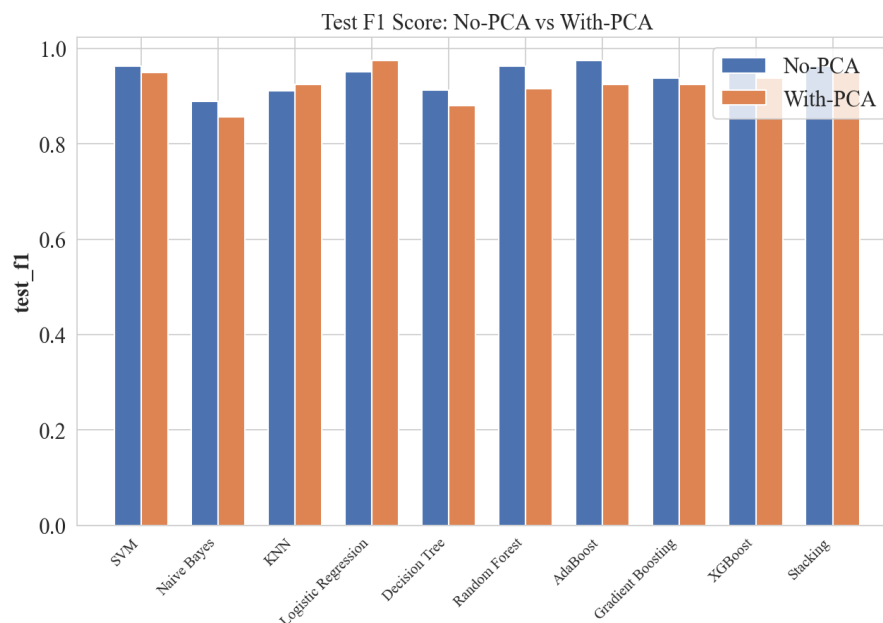


Figure 7: Test F1 Score: No-PCA vs With-PCA

Inference: The test F1-score comparison mirrors the CV accuracy trend – Logistic Regression and KNN benefit from PCA, while Random Forest, AdaBoost, Decision Tree, and Naive Bayes see the largest F1 drops with PCA, reinforcing that linear/distance-based models tolerate (or benefit from) dimensionality reduction better than tree-based ensembles on this dataset.

9 Result Visualization (Selected Models: SVM and Stacking)

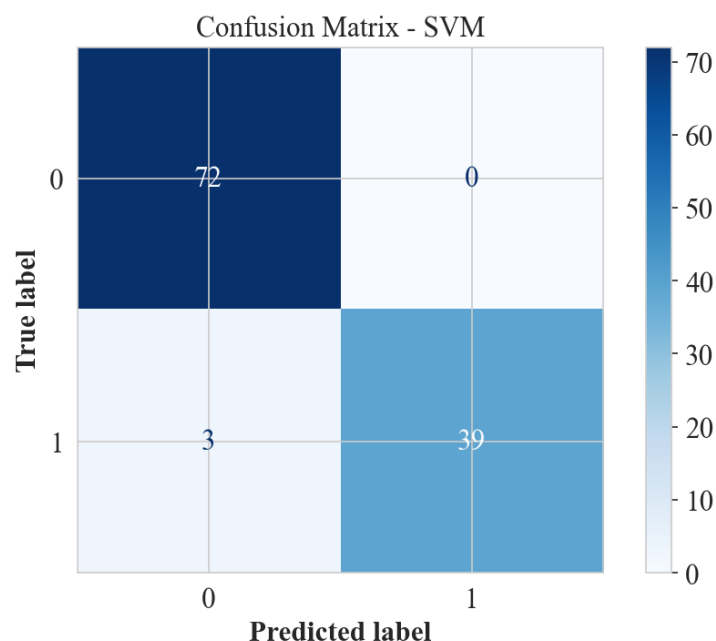


Figure 8: Confusion Matrix – SVM (No-PCA)

Inference: SVM without PCA misclassifies only 3 out of 114 test samples, showing that the original 30-dimensional feature space provides SVM's RBF kernel with a very clean margin between malignant and benign classes.

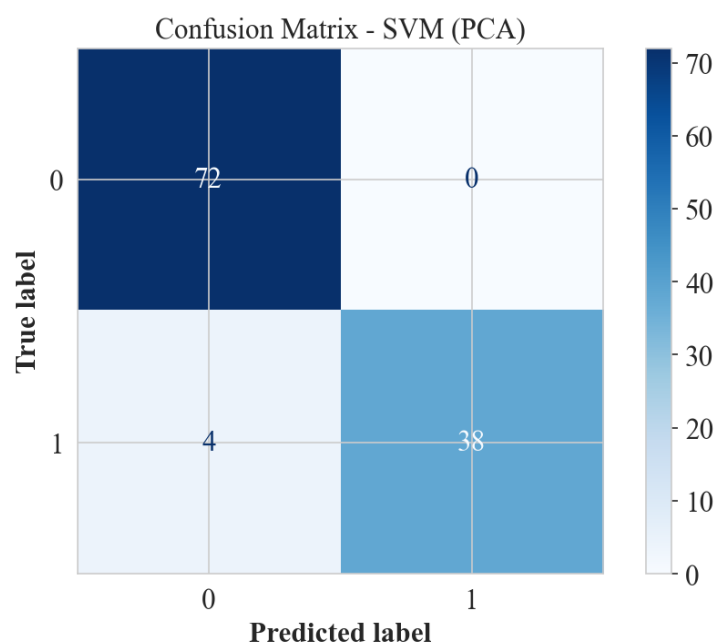


Figure 9: Confusion Matrix – SVM (With-PCA)

Inference: SVM with PCA shows a slightly higher error count than without PCA, consistent with its marginal accuracy drop ($97.37\% \rightarrow 96.49\%$), suggesting the 5% variance discarded by PCA contained a small amount of class-discriminative information useful to the kernel.

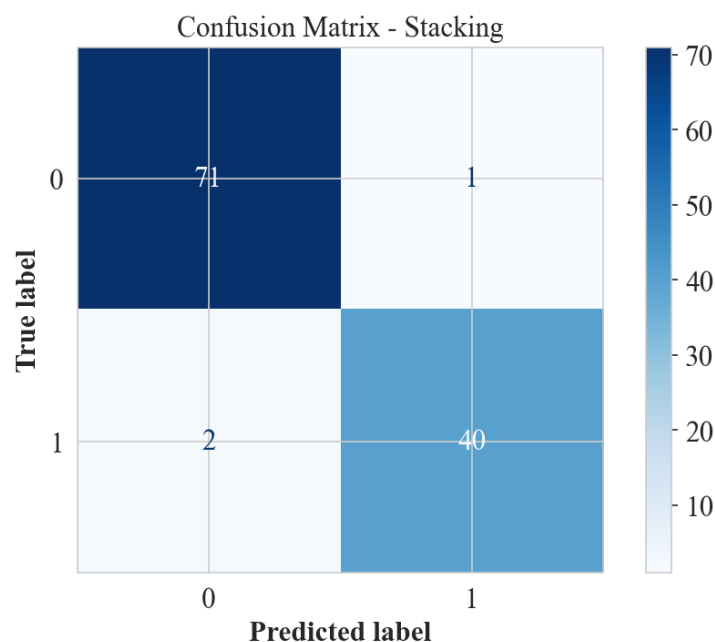


Figure 10: Confusion Matrix – Stacking (No-PCA)

Inference: The Stacking ensemble without PCA achieves strong, balanced performance (96.39% F1), benefiting from its diverse base learners (SVM, Naive Bayes, Decision Tree) each seeing the full original feature space.

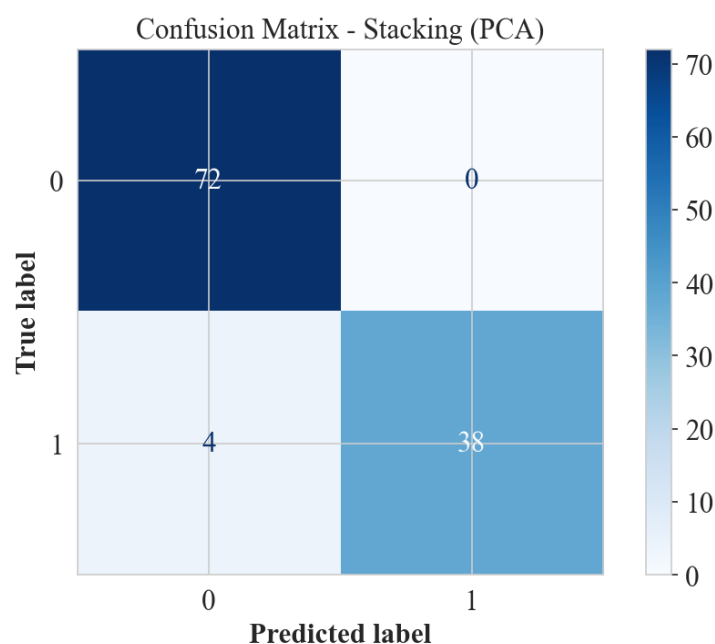


Figure 11: Confusion Matrix – Stacking (With-PCA)

Inference: Stacking with PCA shows a small increase in errors versus without PCA, mirroring SVM’s pattern – since Stacking’s base learners include SVM and Decision Tree, both of which individually lose a little accuracy under PCA, the ensemble inherits a similar modest decline.

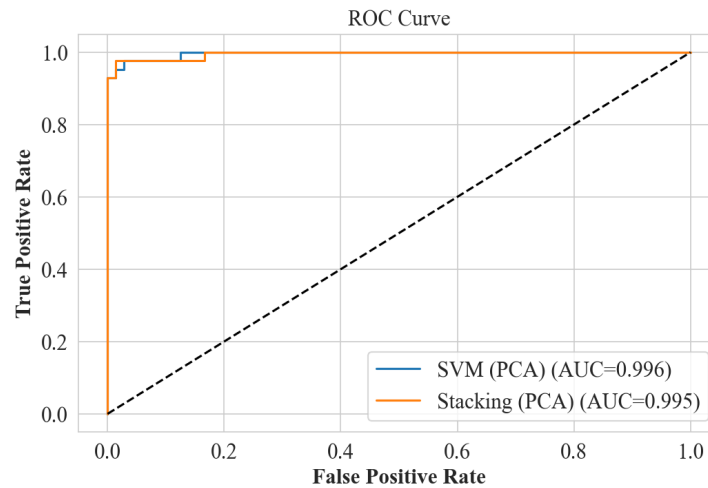


Figure 12: ROC Curve (SVM and Stacking, No-PCA and With-PCA)

Inference: All four curves (SVM/Stacking $\times$ No-PCA/PCA) stay very close to the top-left corner with minimal separation between them, showing that despite small accuracy differences, all four configurations rank malignant vs benign cases with comparably high confidence (ROC-AUC close to 1 in every case).

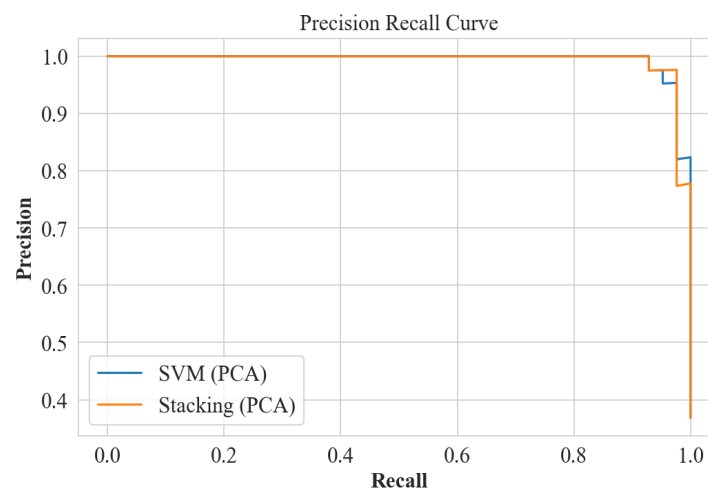


Figure 13: Precision Recall Curve (SVM and Stacking, No-PCA and With-PCA)

Inference: The No-PCA variants of both SVM and Stacking maintain slightly higher precision at high recall than their PCA counterparts, consistent with the small F1 drop observed under PCA for both models.

10 Discussion

PCA's effect on this dataset was model-dependent rather than uniformly positive or negative. Linear/margin-based models (Logistic Regression, SVM) tolerated or even benefited from the 30→10 dimensionality reduction, since PCA directly addresses the multicollinearity visible in the correlation heatmap. In contrast, probabilistic (Naive Bayes) and tree-based models (Decision Tree, Random Forest, Gradient Boosting) performed consistently worse under PCA, since these models rely on the interpretability and independence of individual original features rather than linear combinations of them. Given that PCA reduced accuracy for 6 of the 10 models tested and only clearly helped 2 (Logistic Regression, AdaBoost), and considering the dataset's original 30 features are not prohibitively high-dimensional for the 569 available samples, PCA offers limited practical benefit here beyond a $3\times$ reduction in feature count – useful mainly if training-time or memory constraints made that reduction necessary.

11 Conclusion

On the Wisconsin Diagnostic Breast Cancer dataset, PCA (retaining 10 components for 95% variance) provided a clear benefit only for Logistic Regression and a small benefit for AdaBoost, while modestly hurting Naive Bayes, Decision Tree, Random Forest, Gradient Boosting, and XGBoost. SVM, KNN, and Stacking were largely unaffected either way. The best overall model remained AdaBoost without PCA (98.25% test accuracy, 0.9756 F1), narrowly ahead of SVM/Stacking/Random Forest without PCA (97.37%). PCA is therefore most useful here for linear models sensitive to multicollinearity, and should be applied selectively rather than as a blanket preprocessing step across all model types.

12 References

[1] Scikit-learn: PCA, <https://scikit-learn.org/stable/modules/decomposition.html#pca>.

Scikit-learn: Ensemble Methods, <https://scikit-learn.org/stable/modules/ensemble.html>.

XGBoost Documentation, <https://xgboost.readthedocs.io>.

Wisconsin Diagnostic Breast Cancer Dataset, UCI ML Repository, <https://archive.ics.uci.edu>.